

Prasad Kumkar
 prasad@chainscorelabs.com
 Pune, INDIA

Bitcoin is the peer-to-peer electronic cash protocol and a payment network introduced by Satoshi Nakamoto. This document provides a technical reference and a consolidated knowledge base for the Bitcoin protocol. It is intended to be helpful for Bitcoin-based applications, node clients, and other infrastructure development within the ecosystem. It covers most of the modules implemented by contemporary full nodes: consensus rules, primitive data structures, serialization, peer networking, and relay behavior.

Contents

1	Introduction	4
1.1	Background	4
1.2	Why this specification exists	4
1.3	Related work	4
1.4	Methodology	5
2	Notation	5
2.1	Types and values	5
2.2	Sequences and records	5
2.3	Encoding and byte order	6
2.4	Hashes	6
2.5	Identifiers	6
2.6	Chain context	6
2.7	Cryptographic relations	7
3	Serialization	7
3.1	Fixed-width primitives	7
3.2	CompactSize	7
3.3	Constructors	8
4	Ledger Model	8
4.1	State	8
4.2	Coins	9
4.3	Rule context	9
4.4	Validity classes	10
4.5	Transition relations	10
4.6	Invariants	10
5	Transactions	11
5.1	Identifier and amount primitives	11
5.2	Transaction records	11
5.3	Legacy serialization	12
5.4	Witness serialization	12
5.5	Transaction identifiers	13
5.6	Context-free validity	13
5.7	Coinbase transaction	13
5.8	Value conservation	13
5.9	Locktime and sequence	14
5.10	Signature hashes	14
6	Script Execution	14
6.1	Spend forms	15
6.2	Machine state	15
6.3	Evaluation	16
6.4	Primitive relations	16
6.5	Signature messages	17
6.6	Opcode classes	17

7	Blocks	19
7.1	Block header	19
7.2	Block record	20
7.3	Merkle root	20
7.4	Coinbase transaction	20
7.5	Witness commitment	21
7.6	Context-free block checks	21
7.7	Contextual block checks	21
8	Proof of Work and Chain Selection	22
8.1	Target encoding	22
8.2	Proof-of-work check	23
8.3	Difficulty adjustment	23
8.4	Accumulated work	24
9	Consensus Validation	24
9.1	Validation relation	24
9.2	Header acceptance	24
9.3	Block-data acceptance	25
9.4	Best-chain selection	26
9.5	Block connection	26
9.6	UTXO transition	27
9.7	Rule context	27
9.8	Subsidy and fees	27
9.9	Undo	27
9.10	Reorganization correctness	27
10	Mempool	28
10.1	Entry	28
10.2	Graph Invariants	29
10.3	Admission	29
10.4	Removal	30
10.5	Orphans and Relay View	30
11	Peer-to-Peer Network Protocol	30
11.1	Message envelope	30
11.2	Handshake	31
11.3	Control and negotiation payloads	31
11.4	Service flags	31
11.5	Inventory and object retrieval	32
11.6	Header and block synchronization	32
11.7	Compact block relay	33
11.8	Address relay	33
11.9	Light-client services	34
12	Acknowledgements	35
	Appendix A. Constants	36

Appendix B. Mainnet Parameters	37
B.1 Mainnet parameters	37
B.2 Historical exceptions	37
Appendix C. Remote Procedure Calls (RPC)	38

1 Introduction

1.1 Background

Bitcoin was introduced by Satoshi Nakamoto as a peer-to-peer electronic cash system: a network in which payments can be validated by participants directly, without requiring a financial intermediary to order transactions or prevent double spending [19]. Its core design combines digital signatures, a public append-only history, a proof-of-work timestamp chain, and local validation by independently operated nodes. Transactions spend previously created outputs and create new outputs; blocks commit ordered transaction sets; nodes accept the valid chain with the greatest accumulated proof of work.

The consequence is a protocol whose authority is operational rather than institutional. No server, committee, or publication grants validity to a transaction or block. A block is useful only if economically relevant validating nodes accept it under their consensus rules, and a transaction is final only to the extent that it remains in the accepted proof-of-work history. This makes Bitcoin unusually sensitive to precise definitions: serialization, hash preimages, script evaluation, subsidy, difficulty adjustment, lock-time, witness commitments, and deployment rules are not merely engineering details. They are the boundary between a node that follows the same network and a node that has forked into a different system.

1.2 Why this specification exists

Bitcoin does not have a single normative protocol specification in the style of an Internet RFC. Its design is distributed across the original paper, Bitcoin Improvement Proposals, Bitcoin Core source code, release notes, developer documentation, test vectors, and long-running network practice [4, 5, 6, 7]. That distribution has served Bitcoin well: it keeps proposals reviewable, lets implementations evolve conservatively, and avoids granting authority to a document detached from deployed validation behavior. It also leaves developers with a practical problem. To implement, audit, test, or reason about Bitcoin, they must assemble one protocol from many artifacts that use different levels of precision and often mix consensus, policy, wallet behavior, and operational guidance.

This specification is intended to be the consolidated technical reference for that protocol. It states Bitcoin as data structures, encodings, identifiers, state variables, validation predicates, state transitions, and peer messages. It is not a proposal to change Bitcoin, not a replacement for BIPs, and not an authority over consensus. Where this document conflicts with economically relevant node validation, node validation wins. Its purpose is to make the protocol easier to inspect: one place where the rules needed by independent implementers, reviewers, researchers, and protocol engineers are expressed coherently.

1.3 Related work

The original Bitcoin paper defines the motivating problem and the high-level mechanism: timestamped proof of work over a chain of transaction-containing blocks [19]. BIPs refine that design by documenting concrete changes, conventions, and interoperability standards, including consensus soft forks, peer services, wallet formats, and application-level conventions [16, 4]. Bitcoin Core is the dominant full-node implementation and the principal executable reference for contemporary validation and relay behavior [5, 6]. Developer references and generated API documentation are valuable guides, but they are not a complete formal specification of the network protocol [7].

Other cryptocurrency systems have benefited from standalone protocol specifications. The Zcash protocol specification separates high-level protocol concepts from concrete encodings and upgrade-specific consensus rules [33]. Ethereum’s Yellow Paper presents Ethereum as a transaction-based state machine with explicit state-transition functions [25]. The JAM graypaper follows the same tradition of a single coherent formal model for an independently implementable protocol [24]. This document applies that style to Bitcoin while respecting Bitcoin’s different governance reality: no paper, repository, or committee has authority to redefine the deployed protocol.

1.4 Methodology

The baseline for this draft is Bitcoin Core source, its implemented-BIPs index, and the BIPs cited throughout this document are treated as evidence for contemporary full-node behavior [6]. The specification separates three classes of behavior:

1. Consensus rules: predicates that determine whether blocks and transactions can be accepted into the active chain.
2. Peer and relay rules: network records and local admission behavior used by nodes to exchange blocks, transactions, addresses, and negotiation state.
3. Local implementation behavior: storage layout, indexes, caches, RPC surfaces, wallet policy, user interface behavior, and performance choices that may explain Bitcoin Core but are not themselves protocol requirements.

Only the first two classes belong in the main specification. Implementation details are included only when they expose a protocol boundary or prevent an ambiguous reading of consensus or peer behavior. This keeps the document useful as a source-of-truth reference without turning it into a commentary on one code base.

2 Notation

This section fixes the symbols used across the specification. It defines only shared notation and reusable primitive relations; concrete record layouts, script execution, and signature-message construction are defined in their respective sections.

2.1 Types and values

Notation	Meaning
Bool	Truth values $\{\perp, \top\}$.
Byte	The integers $\{0, \dots, 255\}$.
Bytes	Finite byte strings.
Bytes $[n]$	Byte strings of exactly n bytes.
N , Z	Nonnegative integers and integers.
$x : T$	Value x has type T .
$f : S \rightarrow T$	Total function from S to T .
$f(x) = \perp$	f is undefined, fails, or rejects input x .

2.2 Sequences and records

Notation	Meaning
$[x_0, \dots, x_{n-1}]$	Ordered sequence with zero-based indexing.
$x[i]$	Element at index i .
$ x $	Length of sequence or byte string x .
$x y$	Concatenation of sequences or byte strings.

Hex strings are written in display order. Sections that describe serialized fields state whether display order differs from wire order.

Record tables define field order, type, and meaning. Unless explicitly marked as logical state, a record table gives the order used by E_T .

2.3 Encoding and byte order

For a type or record T , $E_T(x)$ is the canonical byte encoding of x as T . The partial inverse $E_T^{-1}(b)$ parses bytes b as T and is undefined for non-canonical or invalid encodings. When parsing succeeds, $E_T^{-1}(E_T(x)) = x$.

$\text{le}(b)$ is the unsigned little-endian integer represented by byte string b . Fixed-width integer encodings and CompactSize are defined in Section 3.

2.4 Hashes

Hash values are protocol bytes. Human display strings often reverse the bytes of 256-bit hashes.

$$\begin{aligned}
H(b) &= \text{SHA256}(\text{SHA256}(b)), \\
\text{HASH160}(b) &= \text{RIPEMD160}(\text{SHA256}(b)), \\
\text{TagHash}_{tag}(b) &= \text{SHA256}(\text{SHA256}(tag)||\text{SHA256}(tag)||b).
\end{aligned}$$

H is `hash256` in Bitcoin source material. Taproot and BIP340 Schnorr signatures use `TagHash` [19, 30, 31].

For a nonempty list of 32-byte hashes L , $\text{MR}(L)$ is the Merkle root obtained by repeatedly replacing adjacent pairs (a, b) with $H(a||b)$. If a level has odd length, the final hash is duplicated for that level. $\text{MR}([]) = \perp$.

2.5 Identifiers

$$\begin{aligned}
txid(tx) &= H(E_0(tx)), \\
wtxid(tx) &= \begin{cases} H(E_w(tx)), & \text{witness serialization is used,} \\ txid(tx), & \text{otherwise,} \end{cases} \\
bh(h) &= H(E_{\text{BlockHeader}}(h)).
\end{aligned}$$

2.6 Chain context

For a chain C , $C[k]$ is the block at height k , with $C[0]$ as genesis. $\text{height}(B)$ is the height of block B , and $\text{tip}(C) = C[|C| - 1]$. $\text{MTP}(C, k)$ is the median timestamp of $C[k]$ and its ten nearest ancestors, or all available ancestors when $k < 10$.

2.7 Cryptographic relations

All public-key signature relations use the secp256k1 group. $\text{ECDSAVerify}(Q, m, \sigma)$ is true iff ECDSA signature σ verifies digest m under public key Q , subject to active script encoding rules. $\text{SchnorrVerify}(Q, m, \sigma)$ is the BIP340 verification relation [28, 30].

3 Serialization

Bitcoin protocol objects are byte strings with fixed field order. Unless a field table states otherwise, E_T serializes integers in little-endian byte order, vectors with a CompactSize element count, and byte vectors with a CompactSize byte length followed by exactly that many bytes.

3.1 Fixed-width primitives

Primitive	Width	Serialization rule
<code>uint8</code> / <code>int8</code>	1 byte	Byte value as stored.
<code>uint16</code> / <code>int16</code>	2 bytes	Least significant byte first.
<code>uint32</code> / <code>int32</code>	4 bytes	Least significant byte first.
<code>uint64</code> / <code>int64</code>	8 bytes	Least significant byte first.
<code>byte[n]</code>	n bytes	Exactly the given bytes, in index order.
<code>bool</code>	1 byte	Zero is false; nonzero is true unless a field table restricts the value.
<code>uint160</code>	20 bytes	160-bit hash value.
<code>uint256</code>	32 bytes	256-bit hash value. Display hex commonly reverses these bytes.

Fixed-width integer fields are serialized in little-endian byte order unless a field table explicitly states otherwise [6].

3.2 CompactSize

Bitcoin transaction and block structures use CompactSize integers to encode counts and byte-vector lengths. CompactSize is canonical: a decoder rejects a value encoded with a longer form than necessary.

Value range	Encoding	Canonicity rule
$0 \leq n < 253$	one byte: n	First byte is less than 0xfd.
$253 \leq n \leq 0xffff$	0xfd followed by <code>uint16</code>	Decoded value must be at least 253.
$2^{16} \leq n \leq 2^{32} - 1$	0xfe followed by <code>uint32</code>	Decoded value must be at least 0x10000.
$2^{32} \leq n \leq 2^{64} - 1$	0xff followed by <code>uint64</code>	Decoded value must be at least 0x100000000.

A parser rejects non-canonical CompactSize encodings and any decoded count or length that exceeds the enclosing field’s rule.

3.3 Constructors

Constructor	Serialized form	Used for
<code>vector<T></code>	<code>compactSize count T[0]</code> <code> ... T[count-1]</code>	Transaction inputs, outputs, block transactions, inventory vectors, and payload lists.
<code>varbytes</code>	<code>compactSize length </code> <code>byte[length]</code>	Scripts, signatures, public keys, witness items, and variable network fields.
<code>string</code>	<code>compactSize length </code> <code>byte[length]</code>	P2P user-agent and payload strings. Character validity is context-specific.
<code>witnessStack</code>	<code>compactSize itemCount </code> <code>varbytes[0] ...</code>	One witness stack for one transaction input.

Scripts, signatures, public keys, witness items, and network payload fields reuse `varbytes` in different contexts. Parsing bytes into opcodes, signatures, keys, or scripts is a later semantic step.

For transactions and blocks, $E_0(x)$ is the serialization with witness data stripped, and $E_w(x)$ is the serialization including witness data where such data is defined. Define:

$$strippedSize(x) = |E_0(x)|, \quad totalSize(x) = |E_w(x)|.$$

The consensus weight formula is:

$$weight(x) = 3 \cdot strippedSize(x) + totalSize(x).$$

For relay and fee accounting, virtual transaction size is the integer ceiling of weight divided by witness scale:

$$vsize(tx) = \left\lceil \frac{weight(tx) + WITNESS_SCALE_FACTOR - 1}{WITNESS_SCALE_FACTOR} \right\rceil.$$

Block consensus enforces `MAX_BLOCK_WEIGHT` against `weight(block)` [17, 6].

4 Ledger Model

Bitcoin validation is modeled as deterministic state transition over a proof-of-work header graph, available block data, and the active UTXO set. This section fixes the shared relations used by later sections.

4.1 State

A full-validation node’s protocol state is:

$$S = (C, H, B, U, Q, \Pi)$$

Symbol	Type	Meaning
C	ActiveChain	Ordered sequence of connected blocks from genesis to the active tip.
H	HeaderIndex	Known block-header graph keyed by block hash, including height, predecessor, target, timestamp, and accumulated work.
B	BlockDataSet	Full block data known to the node, keyed by block hash.
U	UTXOSet	Map from unspent outpoints to coin records at the active-chain tip.
Q	Mempool	Locally accepted unconfirmed transactions. This is relay state, not consensus state.
Π	PeerSet	Connected-peer state and negotiated network capabilities.

Consensus validation reads C, H, B, U and mutates only C, H, B, U . Mempool and peer transitions read consensus state but do not define block validity.

4.2 Coins

The UTXO set maps outpoints to coins:

$$U[(txid, n)] = (value, scriptPubKey, height, coinbase).$$

Coin field	Type	Meaning
value	Amount	Output amount in satoshis. Consensus-valid values are in the money range.
scriptPubKey	Script	Locking script committed by the creating transaction output.
height	uint32	Active-chain height of the creating transaction.
coinbase	bool	True iff the creating transaction is the block coinbase.

4.3 Rule context

Let N be the selected network parameter set. For candidate height h , candidate block time t , and active chain C , define:

$$R = \text{Rules}(N, C, h, t).$$

R is the set of consensus rules active for the candidate object. It includes network constants, buried activation heights, versionbits state, timestamp rules, and script flags. Later sections use R

rather than restating whether P2SH, BIP34, strict DER, locktime, sequence locks, SegWit, Taproot, Tapscript, or network-specific timewarp mitigations are active.

4.4 Validity classes

Class	Meaning
$\text{DecodeValid}_T(b)$	$E_T^{-1}(b)$ succeeds and consumes a canonical encoding of type T .
$\text{FormValid}(x)$	Object x is internally well-formed independent of chain state.
$\text{ContextValid}_R(x, C')$	Object x satisfies height, time, and active-rule predicates under R .
$\text{StateValid}_R(x, U)$	Object x is valid against the current spendable coin set.
$\text{RelayValid}_P(x, Q, U)$	Object x is accepted by local relay policy. This is not a consensus predicate.

4.5 Transition relations

For a non-coinbase transaction, the transaction transition is:

$$\text{ApplyTx}(R, U, tx) = \begin{cases} U', & \text{if } tx \text{ is valid against } U \text{ under } R, \\ \perp, & \text{otherwise.} \end{cases}$$

For a block B_h at height h , block connection is:

$$\text{ConnectBlock}(R, (C, U), B_h) = \begin{cases} (C', U'), & \text{if every block and transaction predicate succeeds,} \\ \perp, & \text{otherwise.} \end{cases}$$

Header acceptance, block-data acceptance, chain selection, and reorganization are the consensus transitions defined in Section 9. Mempool admission is the local transition defined in Section 10.

4.6 Invariants

Every accepted active-chain transition preserves:

Invariant	Statement
Determinism	For fixed N, C, U, B_h , successful connection produces a unique C', U' .
Outpoint uniqueness	At any active tip, each outpoint appears at most once in U .
No double spend	A connected transaction consumes each input outpoint at most once, and a connected block consumes each live outpoint at most once.
Conservation	A non-coinbase transaction cannot create value: $\sum \text{outputs}(tx) \leq \sum \text{spent}(tx)$.
Issuance bound	For each connected block, $\sum \text{outputs}(\text{coinbase}) \leq \text{subsidy}(h) + \sum \text{fees}(B_h)$.

Work selection	The active chain is selected from valid candidates by greatest accumulated work.
----------------	--

5 Transactions

A Bitcoin transaction consumes previously created outputs and creates new outputs. Consensus validation distinguishes the serialized transaction bytes, the identifiers derived from those bytes, and the contextual checks that depend on the UTXO set and block height.

5.1 Identifier and amount primitives

Type	Encoding	Meaning
Txid	uint256	<i>txid(tx)</i> , as defined in Section 2.
Wtxid	uint256	<i>wtxid(tx)</i> , as defined in Section 2.
OutPoint	Txid uint32	Reference to one transaction output by transaction identifier and output index.
Amount	int64	Satoshis. Consensus-valid output amounts satisfy $0 \leq \textit{value} \leq \text{MAX_MONEY}$.

5.2 Transaction records

TxOut			
Field	Type	Encoding	Meaning
value	Amount	int64	Output value in satoshis.
scriptPubKey	Script	varbytes	Locking script.

TxIn			
Field	Type	Encoding	Meaning
previousOutput	OutPoint	36 bytes	Output spent by this input, or the null outpoint for coinbase.
scriptSig	Script	varbytes	Unlocking script or coinbase data.
sequence	uint32	4 little-endian bytes	Finality, replacement, and relative-lock field.

Transaction			
Field	Type	Encoding	Meaning

version	int32	4 little-endian bytes	Transaction version. Versions 1 and 2 are standard by default relay policy; consensus accepts the integer subject to contextual rules.
inputs	vector<TxIn>	CompactSize count, then inputs	Spent outputs or coinbase input.
outputs	vector<TxOut>	CompactSize count, then outputs	Created outputs.
witnesses	vector<Witness>	witness serialization only	One witness stack per input when witness serialization is used.
lockTime	uint32	4 little-endian bytes	Absolute height or time lock.

Witness is `vector<varbytes>`. It is present only in witness serialization and has exactly one stack per input.

5.3 Legacy serialization

$E_0(tx)$, the legacy transaction serialization, is:

$$version||inputs||outputs||lockTime.$$

Part	Encoding	Meaning
version	int32	Transaction version.
inputCount	compactSize	Number of inputs.
inputs	TxIn[inputCount]	Serialized inputs in order.
outputCount	compactSize	Number of outputs.
outputs	TxOut[outputCount]	Serialized outputs in order.
lockTime	uint32	Absolute lock field.

5.4 Witness serialization

$E_w(tx)$, the witness transaction serialization, is:

$$version||marker||flag||inputs||outputs||witnesses||lockTime.$$

Part	Encoding	Meaning
marker	0x00	Distinguishes witness serialization from legacy serialization.
flag	0x01	Indicates witness data follows. Other nonzero flag bits are invalid.

witnesses	Witness[inputCount]	One witness stack per input.
-----------	---------------------	------------------------------

The witness form is valid only when at least one witness stack is nonempty. A transaction with no witness data uses legacy serialization for both Txid and Wtxid [17, 18].

5.5 Transaction identifiers

Identifier	Definition
txid(tx)	Defined in Section 2.
wtxid(tx)	Defined in Section 2.
GenTxid	Pair of identifier kind and hash used by relay to distinguish txid and wtxid announcements.

5.6 Context-free validity

A transaction is context-free valid only if:

1. it has at least one input and at least one output;
2. its stripped size satisfies $strippedSize(tx) \cdot \text{WITNESS_SCALE_FACTOR} \leq \text{MAX_BLOCK_WEIGHT}$;
3. every output value is in range;
4. the sum of output values is in range;
5. no two inputs reference the same previous output;
6. if it is coinbase, it has exactly one input whose previous output is the null outputpoint and whose scriptSig length is between COINBASE_SCRIPTSIG_MIN and COINBASE_SCRIPTSIG_MAX bytes; and
7. if it is not coinbase, no input uses the null outputpoint.

Context-free validity does not prove the inputs exist or signatures are valid; those checks require the UTXO set and rule context [6].

5.7 Coinbase transaction

A coinbase transaction has exactly one input whose previous output is the null outputpoint defined by NULL_TXID and NULL_INDEX. Its input does not spend a UTXO. Its outputs create the block subsidy and collect fees, subject to the coinbase amount rule during block connection.

Coinbase-created outputs require COINBASE_MATURITY confirmations before they may be spent.

5.8 Value conservation

For a non-coinbase transaction:

$$fee(tx) = \sum value(spentInputs(tx)) - \sum value(outputs(tx)).$$

The transaction is invalid if any referenced input is absent, any spent coin is immature coinbase, any output is outside the money range, the output sum is outside the money range, or `fee(tx)` is negative [6].

5.9 Locktime and sequence

`lockTime` values below `LOCKTIME_THRESHOLD` are block heights; values at or above that threshold are Unix times. For a candidate block at height h with rule context R and locktime cutoff t , define the selected lock boundary:

$$\ell(tx, h, t) = \begin{cases} h, & tx.\text{lockTime} < \text{LOCKTIME_THRESHOLD}, \\ t, & tx.\text{lockTime} \geq \text{LOCKTIME_THRESHOLD}, \end{cases}$$

$$\begin{aligned} \text{AbsFinal}(tx, h, t) \iff tx.\text{lockTime} = 0 \vee tx.\text{lockTime} < \ell(tx, h, t) \vee \\ \forall i : tx.\text{inputs}[i].\text{sequence} = \text{SEQUENCE_FINAL}. \end{aligned}$$

If R includes BIP113, $t = \text{MTP}(C, h - 1)$; otherwise t is the candidate block timestamp.

If R includes BIP68, relative locks apply only to version-2-or-higher transactions and only to inputs whose `sequence` does not set `SEQUENCE_LOCKTIME_DISABLE_FLAG`. For validation view V , define $\text{SeqFinal}(tx, C, V, h)$ as follows. If $tx.\text{version} < 2$, it is true. Otherwise, for each applicable input i , let s_i be its sequence value, $v_i = s_i \& \text{SEQUENCE_LOCKTIME_MASK}$, and c_i be the height of the coin spent by input i in V . Height locks require:

$$h > c_i + v_i - 1.$$

When s_i sets `SEQUENCE_LOCKTIME_TYPE_FLAG`, the input is time locked instead. Let $\tau_i = \text{MTP}(C, c_i - 1)$. Time locks require:

$$\text{MTP}(C, h - 1) > \tau_i + v_i \cdot \text{SEQUENCE_LOCKTIME_GRANULARITY} - 1.$$

The transaction satisfies relative locks iff every applicable input satisfies its selected height or time constraint; disabled inputs impose no constraint. If R does not include BIP68, SeqFinal is true [10, 8, 13, 6].

5.10 Signature hashes

Signature validation signs a digest derived from the transaction, input index, spent output, hash type, and active script version. Legacy, SegWit v0, and Taproot/Tapscript spend paths use different digest algorithms, specified in Section 6. These digests are consensus-critical because the signature checks validate exactly those bytes.

6 Script Execution

For each non-coinbase input i , spend authorization is the predicate:

$$\text{Authorize}_R(tx, i, coin, u, H) \in \{\top, \perp\},$$

where $u = (\text{scriptSig}, \text{witness})$ is the unlocking data, $coin$ is the spent UTXO, R is the active rule context, and H is the candidate height and median-time context. Authorization chooses a spend form, constructs a script machine state, and evaluates the committed program.

6.1 Spend forms

Form	Required relation
Legacy	$\text{Eval}_R(\text{scriptSig}, \text{BASE})$ then $\text{Eval}_R(\text{scriptPubKey}, \text{BASE})$; witness empty; final stack top is true.
P2SH	R includes P2SH; scriptSig is push-only; final pushed item is redeemScript ; $\text{HASH160}(\text{redeemScript})$ matches; $\text{Eval}_R(\text{redeemScript}, \text{BASE})$ succeeds on the pushed stack.
Witness v0 key hash	Native witness program length 20; scriptSig empty unless P2SH-wrapped; witness stack is $(\text{signature}, \text{pubkey})$; $\text{HASH160}(\text{pubkey})$ matches; ECDSA verifies M_{wit0} .
Witness v0 script hash	Native witness program length 32; final witness item is witnessScript ; $\text{SHA256}(\text{witnessScript})$ matches; $\text{Eval}_R(\text{witnessScript}, \text{WITNESS_V0})$ succeeds on the remaining witness stack.
Taproot key path	R includes Taproot; witness stack, after optional annex removal, contains one Schnorr signature; BIP340 verifies M_{tr} under the output key.
Taproot script path	Control block commits a tapleaf to the output key. Leaf version 0xc0 evaluates as Tapscript; unknown leaf versions succeed after commitment validation.
Unknown witness version	A syntactically valid witness program with no assigned meaning succeeds without execution.

Native witness spends require empty **scriptSig**; P2SH-wrapped witness spends require **scriptSig** to push exactly the witness program [2, 17, 14, 30, 31, 32].

6.2 Machine state

Script evaluation is a deterministic transition over:

$$X = (tx, i, coin, \text{script}, pc, S, A, C, R, \sigma, k, \alpha, \beta).$$

Field	Meaning
script , pc	Program bytes and instruction cursor.
S, A, C	Main stack, altstack, and conditional-execution stack.
R, σ	Active rule context and signature version: BASE, WITNESS_V0, or TAPSCRIPT.

k	Last executed code-separator position for signature-message construction.
α	Taproot annex, if present.
β	Tapscript validation-weight budget.

Legacy and witness-v0 execution enforce the script limits named in Appendix A. Tapscript removes the script-size and per-script-opcode limits, keeps the stack-item and stack-size limits, and sets:

$$\beta = \text{TAPSCRIPT_SIGOPS_BUDGET_BASE} + \text{serializedWitnessSize}(\text{input}).$$

Each executed signature opcode with a nonempty signature subtracts `TAPSCRIPT_SIGOPS_COST`; negative β fails.

6.3 Evaluation

$$\text{Eval}_R(\text{script}, \sigma, S_0) = \begin{cases} \top, & \text{execution halts successfully with required final stack,} \\ \perp, & \text{parsing, execution, resource, or final-stack validation fails.} \end{cases}$$

If $\sigma = \text{TAPSCRIPT}$, the script is scanned for any `OP_SUCCESSx` index before ordinary execution; encountering one succeeds. Otherwise, instructions are decoded left to right and Step is applied while all entries of C are true. Non-control opcodes in inactive branches do not mutate S , but parse-stage failures and Tapscript `OP_SUCCESSx` detection still apply.

Successful termination requires an empty C . Legacy and witness-v0 require a nonempty S whose top element casts to true. Tapscript requires exactly one stack element, and that element must cast to true.

6.4 Primitive relations

Relation	Definition
<code>CastToBool(x)</code>	False iff x is empty, zero, or negative zero; true otherwise.
<code>Num(x, n, min)</code>	Signed-magnitude little-endian integer from byte vector x , length at most n ; if min , nonzero encodings must be minimal.
<code>MinimalPush(op, x)</code>	True iff op is the shortest permitted push form for x .
<code>LockTime(X, n)</code>	Units match <code>tx.lockTime</code> , $n \leq tx.lockTime$, and the input sequence is nonfinal.
<code>Sequence(X, n)</code>	BIP68 is active in R , disable/type bits permit comparison, and the input sequence satisfies n .
<code>CheckSig(X, sig, key, h)</code>	Empty signatures return false. Legacy and witness-v0 use ECDSA over secp256k1; Taproot and Tapscript use BIP340 Schnorr. Tapscript key size 0 fails, size 32 verifies, and other nonzero key sizes are unknown key types that pass current consensus.

6.5 Signature messages

Signature checks verify exactly one digest:

$$\begin{aligned} M_{\text{legacy}} &= \text{SigMsg}_{\text{legacy}}(tx, i, k, h), \\ M_{\text{wit0}} &= \text{SigMsg}_{\text{wit0}}(tx, i, \text{coin}, k, h), \\ M_{\text{tr}} &= \text{SigMsg}_{\text{tr}}(tx, i, \text{coin}, \alpha, k, h). \end{aligned}$$

Hash type bytes are 0x00 DEFAULT for Taproot, 0x01 ALL, 0x02 NONE, 0x03 SINGLE, and 0x80 ANYONECANPAY ORed with non-default forms. ALL, NONE, and SINGLE select the output commitment set; ANYONECANPAY restricts input commitment to the signed input. Legacy out-of-range SINGLE signs `uint256::ONE`; Taproot out-of-range SINGLE fails [14, 31, 32].

6.6 Opcode classes

Effects use (**before** - **after**) for stack transitions, with the rightmost item at the top of S . Every opcode index is covered by exactly one row below.

Index	Opcode	Effect
0	OP_0 / OP_FALSE	Push empty vector.
1..75	OP_PUSHBYTES_n	Push the next n script bytes.
76	OP_PUSHDATA1	Next <code>uint8</code> gives pushed byte length.
77	OP_PUSHDATA2	Next little-endian <code>uint16</code> gives pushed byte length.
78	OP_PUSHDATA4	Next little-endian <code>uint32</code> gives pushed byte length.
79	OP_1NEGATE	Push -1 .
81..96	OP_1..OP_16	Push integer 1..16.

Index	Opcode	Effect
97	OP_NOP	No effect.
99	OP_IF	($v \neq 0$); push <code>CastToBool(v)</code> to C .
100	OP_NOTIF	($v \neq 0$); push negated condition to C .
101	OP_VERIF	Fail.
102	OP_VERNOTIF	Fail.
103	OP_ELSE	Toggle top of C ; unmatched ELSE fails.
104	OP_ENDIF	Pop top of C ; unmatched ENDIF fails.
105	OP_VERIFY	($v \neq 0$) if true; otherwise fail.
106	OP_RETURN	Fail.

Index	Opcode	Effect
107	OP_TOALTSTACK	Move top of S to A .
108	OP_FROMALTSTACK	Move top of A to S .

109..114	OP_2DROP..OP_2SWAP	Fixed stack rearrangements; fail on underflow.
115	OP_IFDUP	Duplicate top item iff it casts true.
116	OP_DEPTH	Push stack depth.
117..125	OP_DROP..OP_TUCK	Fixed stack rearrangements; fail on underflow.
130	OP_SIZE	$(x - x \text{ len}(x))$.

Index	Opcode	Effect
135	OP_EQUAL	Byte-vector equality.
136	OP_EQUALVERIFY	Equality followed by VERIFY.
139..148	OP_1ADD..OP_SUB	Unary or binary arithmetic over $\text{Num}(x, 4, \min)$.
154..165	OP_BOOLAND..OP_WITHIN	Boolean and numeric comparisons over $\text{Num}(x, 4, \min)$.

Index	Opcode	Effect
166	OP_RIPEMD160	$(x - \text{RIPEMD160}(x))$.
167	OP_SHA1	$(x - \text{SHA1}(x))$.
168	OP_SHA256	$(x - \text{SHA256}(x))$.
169	OP_HASH160	$(x - \text{HASH160}(x))$.
170	OP_HASH256	$(x - \text{H}(x))$.
171	OP_CODESEPARATOR	Set k to this opcode position.
172	OP_CHECKSIG	$(\text{sig key} - \text{ok})$ using CheckSig.
173	OP_CHECKSIGVERIFY	CHECKSIG followed by VERIFY.
174	OP_CHECKMULTISIG	$(\text{dummy sig}[m] \ m \ \text{key}[n] \ n - \text{ok})$; enforces historical dummy rule when active; disabled in Tapscript.
175	OP_CHECKMULTISIGVERIFY	CHECKMULTISIG followed by VERIFY; disabled in Tapscript.
186	OP_CHECKSIGADD	Tapscript only: $(\text{sig } n \ \text{key} - n + \text{ok})$.

Index	Opcode	Effect
176	OP_NOP1	No effect.
177	OP_CHECKLOCKTIMEVERIFY / OP_NOP2	If active in R , require $\text{LockTime}(X, n)$; otherwise no effect.
178	OP_CHECKSEQUENCEVERIFY / OP_NOP3	If active in R , require $\text{Sequence}(X, n)$; otherwise no effect.
179..185	OP_NOP4..OP_NOP10	No effect unless redefined by active soft-fork rules.

Index	Opcode	Effect
80	OP_RESERVED / OP_SUCCESS80	Reserved failure if executed outside Tapscript; Tapscript success.
98	OP_VER / OP_SUCCESS98	Reserved failure if executed outside Tapscript; Tapscript success.
126..129	OP_CAT..OP_RIGHT / OP_SUCCESSx	Disabled failure outside Tapscript; Tapscript success.
131..134	OP_INVERT..OP_XOR / OP_SUCCESSx	Disabled failure outside Tapscript; Tapscript success.
137..138	OP_RESERVED1..OP_RESERVED2 / OP_SUCCESSx	Reserved failure if executed outside Tapscript; Tapscript success.
141..142	OP_2MUL..OP_2DIV / OP_SUCCESSx	Disabled failure outside Tapscript; Tapscript success.
149..153	OP_MUL..OP_RSHIFT / OP_SUCCESSx	Disabled failure outside Tapscript; Tapscript success.
187..254	OP_SUCCESS187..OP_SUCCESS254	Tapscript success; invalid outside Tapscript.
255	OP_INVALIDOPCODE	Fail.

Consensus script behavior is defined by the active P2SH, DER, locktime, sequence, NULLDUMMY, SegWit, Taproot, and Tapscript rules [22, 28, 10, 8, 13, 15, 31, 32, 6].

7 Blocks

A block is a header plus an ordered transaction vector. The header commits to the previous block header, transaction Merkle root, timestamp, target encoding, and nonce. The transaction vector begins with exactly one coinbase transaction.

7.1 Block header

A serialized `BlockHeader` is exactly 80 bytes in the field order shown:

BlockHeader			
Field	Type	Encoding	Meaning
version	int32	4 little-endian bytes	Block version. Version bits use the high-bit pattern defined by BIP 9.
previous BlockHash	uint256	32 bytes	Double-SHA256 hash of the previous block header in protocol byte order.
merkleRoot	uint256	32 bytes	Merkle root of transaction identifiers. Witness data is committed separately.
time	uint32	4 little-endian bytes	Miner-supplied Unix timestamp.

<code>bits</code>	<code>uint32</code>	4 little-endian bytes	Compact proof-of-work target encoding.
<code>nonce</code>	<code>uint32</code>	4 little-endian bytes	Nonce varied during proof-of-work search.

For a block B , $\text{bh}(B) = \text{bh}(B.\text{header})$.

7.2 Block record

Block			
Field	Type	Encoding	Meaning
<code>header</code>	<code>BlockHeader</code>	80 bytes	Serialized block header.
<code>txCount</code>	<code>compactSize</code>	<code>CompactSize</code>	Number of serialized transactions.
<code>transactions</code>	<code>vector<Transaction></code>	transaction encodings in order	First transaction must be coinbase; all later transactions must not be coinbase.

When witness data is present, block serialization includes each transaction’s witness serialization for relay and block-weight computation. The transaction identifier Merkle root continues to use `txid`, not `wtxid`.

7.3 Merkle root

For a block transaction vector $[tx_0, \dots, tx_{n-1}]$, define:

$$L = [\text{txid}(tx_0), \text{txid}(tx_1), \dots, \text{txid}(tx_{n-1})].$$

A block commits to $\text{MR}(L)$. A block with empty L is invalid.

During root computation, a node rejects the duplicated-subtree mutation case: if two identical hashes are paired at a level before an odd-last duplication step, the block is invalid as mutated [6].

7.4 Coinbase transaction

The first transaction in each block is the coinbase transaction. It is the only transaction permitted to have a null prevout. No other transaction in a block may be coinbase. The coinbase creates the block subsidy and collects transaction fees; its total output value must not exceed subsidy plus fees when the block is connected.

$\text{Rules}(N, C, h, t)$ determines whether the candidate height must be serialized at the beginning of the coinbase `scriptSig` [3, 6].

7.5 Witness commitment

If $R = \text{Rules}(N, C, h, t)$ includes SegWit and any transaction has witness data, the coinbase must commit to the witness Merkle root. The witness tree leaves are:

$$W = [0^{256}, wtxid(tx_1), \dots, wtxid(tx_{n-1})].$$

The coinbase input witness contains a 32-byte reserved value r . The witness commitment is:

$$wc = H(\text{MR}(W) || r).$$

The commitment appears in a coinbase output whose script begins with the SegWit commitment header. If multiple outputs match, the highest-index matching output is used [17].

7.6 Context-free block checks

Check	Rule
Transaction count	The block contains at least one transaction.
Transaction count weight	<code>txCount * WITNESS_SCALE_FACTOR <= MAX_BLOCK_WEIGHT</code> .
Coinbase position	Transaction zero is coinbase; no later transaction is coinbase.
Merkle root	Header <code>merkleRoot</code> equals $\text{MR}(L)$ and is not mutated.
Stripped size	<code>strippedSize(block) * WITNESS_SCALE_FACTOR <= MAX_BLOCK_WEIGHT</code> .
Transaction validity	Every transaction passes context-free transaction checks.
Legacy sigops	<code>legacySigOps(block) * WITNESS_SCALE_FACTOR <= MAX_BLOCK_SIGOPS_COST</code> .

7.7 Contextual block checks

For a candidate block at height h extending chain C , the previous block is $C[h - 1]$, and $R = \text{Rules}(N, C, h, \text{time})$.

Check	Rule
Previous header	The previous header exists and is valid for extension.
Median time	<code>time</code> is greater than $\text{MTP}(C, h - 1)$.
Future time	A node does not accept the block while <code>time</code> is more than <code>MAX_FUTURE_BLOCK_TIME</code> after the node's current clock time; this is temporary future status, not permanent consensus invalidity.
BIP94 timewarp	On networks enforcing BIP94, the first block of each difficulty-adjustment interval except genesis must have <code>time >= previous.time - BIP94_TIMEWARP_GRACE</code> .
Difficulty	<code>bits</code> encodes the target required for the height and network.

Finality	All transactions satisfy $\text{AbsFinal}(tx, h, t)$, where $t = \text{MTP}(C, h - 1)$ if R includes BIP113 and the candidate block time otherwise.
Coinbase height	If R includes BIP34, the block commits to the correct height in the coinbase.
Witness commitment	If R includes SegWit and the block has witness data, the block contains the correct coinbase witness commitment.
Weight	Block weight is at most <code>MAX_BLOCK_WEIGHT</code> .

The timestamp checks above follow the active validation predicates for median time past, future-time acceptance, and BIP94 timewarp mitigation [6, 12].

8 Proof of Work and Chain Selection

Bitcoin blocks commit to proof of work through the block header hash. A block is valid only if its header hash satisfies the target encoded by the header’s `nBits` field.

8.1 Target encoding

The `nBits` field is a 32-bit compact target. For compact value c , let:

$$e = c \gg 24, \quad m = c \& 0x007fffff, \quad s = c \& 0x00800000.$$

The decoded target $C^{-1}(c)$ is:

$$C^{-1}(c) = \begin{cases} m \gg 8(3 - e), & e \leq 3, \\ m \ll 8(e - 3), & e > 3. \end{cases}$$

The sign bit is inherited from an older multiple-precision integer format and is not permitted for proof-of-work targets. A decoded target is invalid if any of the following conditions holds:

- s is set and $m \neq 0$;
- $C^{-1}(c) = 0$;
- decoding overflows 256 bits, which occurs when $m \neq 0$ and either $e > 34$, or $m > 0xfff$ and $e > 33$, or $m > 0xffff$ and $e > 32$;
- $C^{-1}(c)$ is greater than the network’s proof-of-work limit.

This compact-target decoding matches contemporary Bitcoin Core consensus validation [6].

The compact encoding $C(T)$ of target T is computed as follows. Let `size` = `ceil(bitLength(T) / 8)`. If `size` <= 3, set `word` = `T` << `8(3 - size)`; otherwise set `word` = `T` >> `8(size - 3)`. If `word` has bit `0x00800000` set, right-shift `word` by 8 and increment `size`. Then:

$$C(T) = (size \ll 24) | (word \& 0x007fffff).$$

Consensus proof-of-work targets are nonnegative, so the sign bit is not set in valid block targets.

8.2 Proof-of-work check

For header h , let:

$$T_h = C^{-1}(h.\text{bits}), \quad p_h = \text{le}(\text{bh}(h)).$$

A candidate block satisfies proof of work iff T_h is a valid target and:

$$p_h \leq T_h.$$

A node rejects the block header if target decoding fails or if the hash integer is greater than the target. The network-specific proof-of-work limit is part of the chain parameters, not a value inferred from the genesis block [6].

8.3 Difficulty adjustment

Mainnet difficulty adjusts over fixed-height intervals. Test networks can have different adjustment behavior. Network-specific exceptions must be cited from chain parameter sources rather than inferred from mainnet behavior.

For mainnet:

$$\text{DIFFICULTY_ADJUSTMENT_INTERVAL} = \frac{\text{TARGET_TIMESPAN}}{\text{TARGET_SPACING}}.$$

A candidate block's `nBits` must equal the value returned by the network's difficulty-adjustment function for the previous block and candidate timestamp [6].

For networks without the minimum-difficulty exception, the algorithm is:

1. If the candidate height is not an exact difficulty-adjustment interval, return the previous block's `nBits`.
2. Otherwise, identify the first block in the previous adjustment period. For mainnet this is the ancestor `DIFFICULTY_ADJUSTMENT_INTERVAL - 1` blocks before `previous`.
3. Compute $\Delta = \text{time}(\text{previous}) - \text{time}(\text{first})$.
4. Clamp Δ to the range

$$\frac{\text{TARGET_TIMESPAN}}{\text{MAX_RETARGET_FACTOR}} \leq \Delta \leq \text{TARGET_TIMESPAN} \cdot \text{MAX_RETARGET_FACTOR}.$$

5. Decode the previous period's base target. On testnet4-like networks with BIP94 enforcement, the base target is taken from the first block of the period; otherwise it is taken from the previous block.
6. Set:
$$T' = \frac{\text{baseTarget} \cdot \Delta}{\text{TARGET_TIMESPAN}}.$$
7. If T' exceeds the network proof-of-work limit, use the proof-of-work limit.
8. Return $C(T')$.

On no-retarget networks, the function returns the previous block’s `nBits`. On networks that allow special minimum-difficulty blocks, the function may return the proof-of-work limit when the candidate timestamp is more than $2 \cdot \text{TARGET_SPACING}$ after the previous block; otherwise it searches backward for the most recent non-special difficulty in the current adjustment period [6].

8.4 Accumulated work

Nodes compare competing valid chains by accumulated work rather than by block count. This follows the proof-of-work security model described in the Bitcoin whitepaper [19].

For each accepted header, the node adds the work implied by that header’s target to the predecessor’s accumulated work. Headers and blocks with invalid proof of work are rejected before they can contribute to active-chain selection.

For a valid compact target, Bitcoin Core computes the work represented by a header as:

$$\text{work}(c) = \left\lfloor \frac{2^{256} - C^{-1}(c) - 1}{C^{-1}(c) + 1} \right\rfloor + 1.$$

This is algebraically equivalent to $\lfloor 2^{256} / (C^{-1}(c) + 1) \rfloor$ while avoiding construction of the unrepresentable integer 2^{256} in a 256-bit integer type [6]. The active chain is selected from valid candidates by greatest accumulated work, not by height.

9 Consensus Validation

Consensus validation determines whether a node may accept a chain history as valid. It instantiates the transition relations from Section 4 for headers, block data, active-chain connection, and reorganization.

9.1 Validation relation

Let N be the network parameter set. For candidate block b , candidate height h , candidate block time t , active chain C , and UTXO set U , let $R = \text{Rules}(N, C, h, t)$. A successful connection produces:

$$(C, U) \xrightarrow{b} (C', U')$$

where C' extends or reorganizes C to include b , and U' is the UTXO set obtained after applying every transaction in the connected branch. If any consensus check fails, the active pair (C, U) is unchanged.

9.2 Header acceptance

A header is accepted into the header graph only if all conditions below hold. Header acceptance does not imply that block transaction data is known or valid.

Condition	Required rule
Uniqueness	If $\text{bh}(h)$ is already known, the existing header state is reused.

Previous header	Every non-genesis header references a known previous header.
Proof of work	$\text{le}(\text{bh}(h)) \leq C^{-1}(h.\text{bits})$.
Target	$C^{-1}(h.\text{bits})$ is valid for the height, network, and difficulty-adjustment interval.
Timestamp	The header time is greater than the median timestamp of the previous block and up to ten ancestors. A header whose time is more than <code>MAX_FUTURE_BLOCK_TIME</code> after the node's current clock time is future-dated and not eligible for active validation until time advances.
BIP94 timewarp	On networks enforcing BIP94, a difficulty-adjustment block other than genesis has time at least <code>previous.time - BIP94_TIMEWARP_GRACE</code> .
Rule context	R permits the candidate height, time, version field, and network-specific activation state.
Invalid ancestry	A header descending from a known-invalid header is not eligible for best-chain selection.

For each accepted header, the node records height, predecessor, $C^{-1}(h.\text{bits})$, $\text{bh}(h)$, and accumulated work. Accumulated work is the sum of the work represented by each header from genesis through that header.

9.3 Block-data acceptance

Full block data is accepted only after the corresponding header is accepted. The following checks are performed before block data can become a candidate for active-chain connection:

Check	Required rule
Block form	The block has at least one transaction, satisfies the stripped-size, transaction-count, and weight limits in Section 7, and the first transaction is the only coinbase transaction.
Merkle root	Header <code>merkleRoot</code> equals $\text{MR}(L)$ for the block's txid leaves, and the mutated-root condition is rejected.
Transaction form	Every transaction passes context-free transaction checks.
Legacy sigops limit	<code>legacySigOps(block) * WITNESS_SCALE_FACTOR</code> $\leq$ <code>MAX_BLOCK_SIGOPS_COST</code> before P2SH and witness sigops are counted during connection.
Contextual finality	Every transaction satisfies <code>AbsFinal</code> for the candidate block height and locktime cutoff.
Coinbase height	If R includes BIP34, the coinbase script begins with the candidate height.
Witness commitment	If R includes SegWit and the block has witness data, the coinbase witness commitment is present and correct.
Weight	The block weight is at most <code>MAX_BLOCK_WEIGHT</code> .

9.4 Best-chain selection

Among branches whose headers are valid and whose required block data is available, the selected branch is the valid branch with greatest accumulated work. A candidate branch is not eligible while any block on the path from the active fork to the candidate tip is missing block data or is known invalid.

1. Choose the highest-work eligible candidate tip.
2. Find the fork point with the current active chain.
3. Disconnect active-chain blocks back to the fork.
4. Connect candidate-branch blocks forward in height order.
5. If a candidate block is consensus-invalid, exclude that block and its descendants from future selection and retry with the next candidate.

9.5 Block connection

Connecting a non-genesis block applies the block's transactions to a temporary UTXO view whose best block is the previous block hash. The view becomes active only after all checks below pass.

Rule	Required behavior
BIP30 no-overwrite	A transaction output must not overwrite an existing unspent outputpoint except for the historical BIP30 exceptions and buried rules that make the check unnecessary after BIP34.
Input availability	Every non-coinbase input references an unspent coin in the candidate view.
Duplicate spends	A transaction must not spend the same outputpoint more than once.
Coinbase maturity	Coinbase-created outputs may be spent only after <code>COINBASE_MATURITY</code> confirmations.
Sequence locks	Every non-coinbase transaction satisfies the BIP68 relative-lock predicate from Section 5.
Value range	Every output and every sum used during validation remains in the money range.
Value conservation	For each non-coinbase transaction, input value is at least output value; the difference is the transaction fee.
Script validity	Each input satisfies the previous output's locking script under the script flags active for the block.
Sigops cost	Total block signature-operation cost is at most <code>MAX_BLOCK_SIGOPS_COST</code> .
Subsidy	Coinbase output value is at most block subsidy plus total fees.

The genesis block is special: its coinbase transaction is part of the block history but is not connected as a spendable coin by normal block-connection rules.

9.6 UTXO transition

For a valid non-coinbase transaction tx , let $I(tx)$ be its input outpoints and $O(tx)$ be its outputs indexed by output number. The candidate UTXO set is updated as follows:

$$U_{after} = (U_{before} \setminus I(tx)) \cup \{((txid(tx), n), coin(out_n, height, false)) : out_n \in O(tx)\}.$$

For the coinbase transaction, there are no spent inputs, and created outputs are inserted with `coinbase = true`. Outputs whose value is zero are still outputs and are inserted unless another consensus rule rejects the transaction.

9.7 Rule context

$Rules(N, C, h, t)$ determines the script flags and contextual block predicates used for connection. Mainnet consensus rule context includes the buried and deployed rules for P2SH, strict DER signatures, locktime and sequence verification, SegWit, Taproot, and Tapscrip when included in R [2, 22, 28, 10, 8, 13, 17, 14, 15, 31, 32].

Policy-only flags may be stricter for mempool relay, but policy failure is not block invalidity.

9.8 Subsidy and fees

The block subsidy is determined by height and network parameters. For mainnet, the subsidy starts at 50 BTC and halves every 210,000 blocks until it reaches zero by integer right shift [27]. The consensus rule for a connected block is:

$$\sum outputs(coinbase) \leq subsidy(height) + \sum fees(noncoinbase).$$

Fees are computed from connected inputs and outputs. A transaction whose output sum exceeds its input sum is invalid.

9.9 Undo

Before applying a non-coinbase transaction, a node records the spent coin for each input in input order. The undo record for a block is the ordered sequence of those per-transaction records, excluding the coinbase. Undo is sufficient to restore the previous UTXO set when the block is disconnected; the byte encoding of undo data is not a consensus rule.

9.10 Reorganization correctness

A reorganization from old tip o to new tip n is valid only if:

1. the branch ending at n has greater accumulated work than the current active branch or is otherwise selected by the same greatest-work rule;
2. every disconnected block is reversed exactly using the UTXO state that existed after that block was connected;
3. every connected block on the new branch passes normal block connection; and

4. after the transition, the active UTXO set equals the result of applying the active chain from genesis to n .

Transactions removed from disconnected blocks reenter relay only under current mempool policy.

10 Mempool

The mempool is local, non-consensus state. It is a node’s current set of unconfirmed transactions that the node is willing to keep, relay, and consider for block-template construction. A transaction can be valid in a block and absent from a node’s mempool.

Let U be the active UTXO set. A mempool is:

$$Q = (M, G, I),$$

where M is a finite map from transaction identifier to mempool entry, G is the dependency graph induced by unconfirmed spends, and I is the set of indexes needed to query entries by transaction identifier, witness transaction identifier, spent outpoint, created outpoint, feerate, and entry time. In equations, $t \in M$ ranges over transactions in the entry map, and $Q \cup \{tx\}$ denotes the mempool obtained by adding tx to M and recomputing G and I . For validation against mempool ancestors, define:

$$\text{View}(Q, U) = U \cup \bigcup_{t \in M} \text{Out}(t).$$

10.1 Entry

Field	Type	Meaning
tx	Transaction	Serialized transaction accepted into the mempool.
txid, wtxid	uint256	Transaction identifiers from Section 5.
fee	int64	Input value minus output value under $\text{View}(Q, U)$.
weight, vsize	int64	Weight and virtual size from Section 3.
sigopsCost	int64	Signature-operation cost under the selected script rules.
spends	OutPoint set	Previous outputs consumed by tx .
creates	OutPoint set	Outputs created by tx .
parents	Txid set	Mempool transactions directly spent by tx .
children	Txid set	Mempool transactions that directly spend tx .
entryTime, entryHeight	local clock, height	Admission time and active-chain height at admission.

The entry fields are logical fields: an implementation may derive, cache, or index them differently, but externally observable mempool behavior must be equivalent.

10.2 Graph Invariants

For a transaction t , let $\text{In}(t)$ be its spent outpoints and $\text{Out}(t)$ be its created outpoints. Direct mempool dependencies are:

$$\begin{aligned}\text{parents}_Q(t) &= \{u \in M : \text{In}(t) \cap \text{Out}(u) \neq \emptyset\}, \\ \text{children}_Q(t) &= \{u \in M : \text{In}(u) \cap \text{Out}(t) \neq \emptyset\}.\end{aligned}$$

A mempool is well-formed relative to U iff:

$$\text{TxAvailable}_Q(t, U) \iff \forall x \in \text{In}(t) : x \in U \cup \bigcup_{u \in M \setminus \{t\}} \text{Out}(u).$$

$$\begin{aligned}\text{WellFormed}(Q, U) &\iff \forall t \in M : \neg \text{Coinbase}(t) \wedge \text{ContextFreeValid}(t) \wedge \text{TxAvailable}_Q(t, U) \wedge \\ &\quad \forall a, b \in M, a \neq b : \text{In}(a) \cap \text{In}(b) = \emptyset \wedge \\ &\quad G = \{(u, t) : u \in \text{parents}_Q(t)\} \wedge \text{Acyclic}(G).\end{aligned}$$

The graph order is the spend order: a parent precedes every child. Any block template or package derived from Q must include transactions in a topological order of G .

10.3 Admission

Admission is a local transition:

$$Q \xrightarrow[P]{tx, U, H} Q',$$

where P is the node's local policy parameter set and H is the next-block height and median-time context. Write $H = (C, h, t)$, where C is the active chain, h is the next block height, and t is the next block locktime cutoff. Let $V_Q = \text{View}(Q, U)$. A single transaction admission succeeds iff:

$$\begin{aligned}\text{Admit}_P(Q, U, H, tx) &\iff tx \notin M \wedge \text{WellFormed}(Q \cup \{tx\}, U) \wedge \\ &\quad \text{ConsensusValid}(V_Q, tx, H) \wedge \text{AbsFinal}(tx, h, t) \wedge \text{SeqFinal}(tx, C, V_Q, h) \wedge \\ &\quad \text{Policy}_P(Q, U, tx, H), \\ Q' = Q \cup \{tx\} &\iff \text{Admit}_P(Q, U, H, tx).\end{aligned}$$

Policy_P includes node-local standardness, feerate, resource, replacement, package, and denial-of-service rules. Those rules do not define consensus validity; when they are specified, they belong with the transaction, script, package, or relay behavior they constrain.

A package π is admitted atomically by applying the same transition to a topologically sorted sequence of transactions. If any member fails, the package transition fails and Q is unchanged.

10.4 Removal

Let B be a connected block and U' the UTXO set after connecting it. Let H' be the next-block context after connecting B . Connecting B removes every included transaction and every mempool transaction that conflicts with the block:

$$\begin{aligned} \text{ConnectBlock}(Q, B, U') = \{t \in M : t \notin B.\text{txs} \wedge \\ \text{In}(t) \cap \text{Spent}(B) = \emptyset \wedge \\ \text{ConsensusValid}(\text{View}(Q, U'), t, H')\}. \end{aligned}$$

After removal, all entry indexes and dependency edges are recomputed so that $\text{WellFormed}(Q, U')$ holds.

Disconnecting a block does not automatically restore its transactions to the mempool. The disconnected transactions become admission candidates and may reenter only through the current admission relation under the post-disconnect chain state.

10.5 Orphans and Relay View

A transaction with unavailable inputs is not a mempool entry. A node may keep an orphan cache O of such transactions and retry admission when parents become available, but O is local resource state and has no consensus meaning.

Transaction relay announces transactions from Q by `txid` or `wtxid`, serves requested transactions if still present, and may filter, delay, or suppress announcements according to local policy. Relay choices do not change the definition of Q [6].

11 Peer-to-Peer Network Protocol

The P2P protocol transports blocks, transactions, headers, addresses, filters, and relay-control messages between nodes. This section specifies interoperable wire behavior.

11.1 Message envelope

Legacy unencrypted Bitcoin P2P messages use a 24-byte header followed by a payload:

Field	Type	Meaning
<code>start</code>	<code>byte[4]</code>	Network magic identifying mainnet, testnet, signet, or regtest.
<code>command</code>	<code>byte[12]</code>	ASCII command name padded with zero bytes.
<code>payloadSize</code>	<code>uint32</code>	Number of payload bytes.
<code>checksum</code>	<code>byte[4]</code>	First four bytes of $H(\text{payload})$.
<code>payload</code>	<code>byte[]</code>	Command-specific serialized payload of length <code>payloadSize</code> .

BIP324 defines an encrypted v2 transport that changes transport framing after negotiation, but command payload semantics remain command-specific [29, 6].

11.2 Handshake

A connection becomes usable after version negotiation:

1. each side sends **version**;
2. each side validates network, protocol version, services, timestamp, nonce, and user-agent limits;
3. each side sends **verack**; and
4. optional relay, compact-block, address-relay, filter, and transport features are negotiated by later messages.

version field	Type	Meaning
version	int32	Protocol version offered by the sender.
services	uint64	Service bits offered by the sender.
timestamp	int64	Sender Unix time in seconds.
addrRecv	network address	Address of the receiving peer as understood by the sender.
addrFrom	network address	Address of the sender.
nonce	uint64	Random connection nonce used to detect self-connections.
userAgent	string	BIP14 user-agent string.
startHeight	int32	Sender's current best height estimate.
relay	bool, optional	BIP37 transaction-inventory relay preference when present.

11.3 Control and negotiation payloads

Message	Payload	Meaning
verack	empty	Completes version negotiation.
sendheaders	empty	Requests direct headers announcements instead of block inventory announcements.
wtxidrelay	empty	Negotiates transaction relay by witness transaction identifier.
sendaddrv2	empty	Negotiates BIP155 address relay.
ping	uint64 nonce	Liveness probe.
pong	uint64 nonce	Response echoing the received ping nonce.
feefilter	uint64 feerate	Asks the peer not to announce transactions below the feerate in satoshis per 1000 virtual bytes.

11.4 Service flags

Flag	Bit	Meaning
------	-----	---------

NODE_NETWORK	0	Peer can serve the full block chain.
NODE_BLOOM	2	Peer supports BIP37 bloom-filter service.
NODE_WITNESS	3	Peer supports witness transaction and block relay.
NODE_COMPACT_FILTERS	6	Peer supports BIP157 compact block filter service.
NODE_NETWORK_LIMITED	10	Peer can serve at least a limited recent block window.

Service flags advertise capability. A node must still validate responses and apply local resource limits.

11.5 Inventory and object retrieval

Inventory relay announces objects by type and hash. A peer requests announced objects with `getdata`; the responder sends the corresponding object or a `notfound` response when unavailable.

Record	Fields	Meaning
<code>InventoryVector</code>	<code>type:uint32</code> , <code>hash:uint256</code>	Identifies a transaction, block, filtered block, compact block, witness transaction, or witness block.
<code>inv</code>	<code>vector<</code> <code>InventoryVector></code>	Announces available objects.
<code>getdata</code>	<code>vector<</code> <code>InventoryVector></code>	Requests announced objects.
<code>notfound</code>	<code>vector<</code> <code>InventoryVector></code>	Reports requested objects not available from the responder.

Witness inventory types request serialization that includes witness data. A peer that has not negotiated witness support must not rely on receiving witness objects from that connection [18].

Inventory type	Value	Object requested or announced
<code>MSG_TX</code>	1	Transaction by <code>txid</code> .
<code>MSG_BLOCK</code>	2	Block without requiring witness serialization.
<code>MSG_FILTERED_BLOCK</code>	3	BIP37 filtered block.
<code>MSG_CMPCT_BLOCK</code>	4	BIP152 compact block.
<code>MSG_WITNESS_TX</code>	$2^{30} + 1$	Transaction including witness data, identified by <code>txid</code> request type with witness flag.
<code>MSG_WITNESS_BLOCK</code>	$2^{30} + 2$	Block including witness data.
<code>MSG_FILTERED_WITNESS_BLOCK</code>	$2^{30} + 3$	Filtered block including witness data.

11.6 Header and block synchronization

Headers synchronize chain work before full block data. A locator is a sequence of block hashes, newest first, stepping back from a known tip toward genesis.

Messages `getheaders` and `getblocks` carry a locator and stop hash. Message `headers` returns block headers in forward order, each followed by `CompactSize` zero. Message `block` carries serialized full block data. A node must validate proof of work, linkage, target, timestamp, and deployment context for received headers before using them for chain selection.

Record	Fields	Encoding rule
<code>BlockLocator</code>	<code>version</code> , <code>hashes</code>	<code>int32</code> version followed by <code>vector<uint256></code> locator hashes.
<code>getheaders/</code> <code>getblocks</code> <code>headers</code> item	<code>version</code> , <code>locator</code> , <code>stopHash</code> <code>header</code> , <code>txCount</code>	<code>int32</code> , <code>locator</code> vector, then <code>uint256</code> stop hash. 80-byte block header followed by <code>CompactSize</code> zero. A nonzero count is invalid for <code>headers</code> .

11.7 Compact block relay

BIP152 compact blocks reduce block-relay bandwidth by sending a header, short transaction identifiers, selected prefilled transactions, and requesting any missing transactions [9].

Message `sendcmpct` negotiates compact-block mode and announcement preference. Messages `cmpctblock`, `getblocktxn`, and `blocktxn` carry the compact block and missing-transaction exchange. Compact reconstruction is an optimization only. The reconstructed full block must pass the same block validation rules as any received `block` message.

Record	Fields	Encoding rule
<code>Compact</code> <code>Block</code>	<code>header</code> , <code>nonce</code> , <code>shortIds</code> , <code>prefilledTx</code>	Header, <code>uint64</code> nonce, <code>CompactSize</code> count plus 6-byte short ids, then <code>vector<PrefilledTx></code> .
<code>PrefilledTx</code>	<code>indexDelta</code> , <code>tx</code>	<code>CompactSize</code> differential index from the previous prefilled transaction, then full transaction.
<code>getblocktxn</code>	<code>blockHash</code> , <code>indexes</code>	Block hash followed by <code>CompactSize</code> differential transaction indexes.
<code>blocktxn</code>	<code>blockHash</code> , <code>transactions</code>	Block hash followed by transaction vector.

11.8 Address relay

Legacy address records and BIP155 address records use different encodings:

Record	Fields	Encoding rule
<code>NetAddress</code> <code>NoTime</code>	<code>services</code> , <code>address</code> , <code>port</code>	<code>uint64</code> services, 16 address bytes, then <code>uint16</code> port in big-endian network byte order. Used inside <code>version</code> .
<code>NetAddress</code> <code>Legacy</code>	<code>time</code> , <code>services</code> , <code>address</code> , <code>port</code>	<code>uint32</code> time plus <code>NetAddressNoTime</code> . Used by <code>addr</code> .

AddrV2	time, services, networkId, address, port	uint32 time, CompactSize services, uint8 network id, CompactSize address length, address bytes, then big-endian port.
--------	--	--

Messages `addr` and `addrv2` relay legacy and BIP155 address records. Message `sendaddrv2` announces BIP155 address-relay support, and `getaddr` requests known peer addresses subject to local rate limits. Address relay is unauthenticated gossip. Received addresses are hints, not validated reachability claims [23].

11.9 Light-client services

Bitcoin P2P includes two optional light-client service families:

Service	Messages	Status
BIP37 bloom filters	<code>filterload</code> , <code>filteradd</code> , <code>filterclear</code> , <code>merkleblock</code>	Optional peer service; disabled unless a node chooses to provide bloom-filter support.
BIP157/158 compact filters	<code>getcfilters</code> , <code>cfilter</code> , <code>getcfheaders</code> , <code>cfheaders</code> , <code>getcfcheckpt</code> , <code>cfcheckpt</code>	Optional compact-filter service for clients that verify filters against block-filter headers.

These services do not change block or transaction validity [11, 20, 21].

Message	Payload fields	Encoding rule
<code>filterload</code>	<code>filter</code> , <code>hashFuncs</code> , <code>tweak</code> , <code>flags</code>	<code>varbytes</code> , <code>uint32</code> , <code>uint32</code> , <code>uint8</code> .
<code>filteradd</code>	<code>data</code>	<code>varbytes</code> .
<code>filterclear</code>	<code>empty</code>	Clears the peer's BIP37 filter state.
<code>merkleblock</code>	<code>header</code> , <code>total</code> , <code>hashes</code> , <code>flags</code>	Block header, <code>uint32</code> total transactions, <code>vector<uint256></code> hashes, and flag bytes.
<code>getcfilters</code>	<code>type</code> , <code>startHeight</code> , <code>stopHash</code>	<code>uint8</code> , <code>uint32</code> , <code>uint256</code> .
<code>cfilter</code>	<code>type</code> , <code>blockHash</code> , <code>filter</code>	<code>uint8</code> , <code>uint256</code> , <code>varbytes</code> .
<code>getcfheaders</code>	<code>type</code> , <code>startHeight</code> , <code>stopHash</code>	<code>uint8</code> , <code>uint32</code> , <code>uint256</code> .
<code>cfheaders</code>	<code>type</code> , <code>stopHash</code> , <code>prevHeader</code> , <code>filterHashes</code>	<code>uint8</code> , two <code>uint256</code> values, then <code>vector<uint256></code> .
<code>getcfcheckpt</code>	<code>type</code> , <code>stopHash</code>	<code>uint8</code> , <code>uint256</code> .
<code>cfcheckpt</code>	<code>type</code> , <code>stopHash</code> , <code>headers</code>	<code>uint8</code> , <code>uint256</code> , <code>vector<uint256></code> .

12 Acknowledgements

Bitcoin protocol was invented by Satoshi Nakamoto and has been advanced, maintained, and secured by generations of Bitcoin Core developers, reviewers, maintainers, researchers, miners, node operators, and users; we thank them for their contributions. This specification is a derivative reference work over their protocol design, implementation, review, and maintenance.

Appendix A. Constants

Constant	Value
COIN	100,000,000 satoshis
MAX_MONEY	21,000,000 · COIN
WITNESS_SCALE_FACTOR	4
MAX_BLOCK_WEIGHT	4,000,000 weight units
MAX_BLOCK_SIGOPS_COST	80,000
COINBASE_MATURITY	100 blocks
SUBSIDY_HALVING_INTERVAL	210,000 blocks
NULL_TXID	0 ²⁵⁶
NULL_INDEX	0xffffffff
COINBASE_SCRIPTSIG_MIN	2 bytes
COINBASE_SCRIPTSIG_MAX	100 bytes
LOCKTIME_THRESHOLD	500,000,000
SEQUENCE_FINAL	0xffffffff
SEQUENCE_LOCKTIME_DISABLE_FLAG	1 $\ll$ 31
SEQUENCE_LOCKTIME_TYPE_FLAG	1 $\ll$ 22
SEQUENCE_LOCKTIME_MASK	0x0000ffff
SEQUENCE_LOCKTIME_GRANULARITY	512 seconds
MAX_SCRIPT_ELEMENT_SIZE	520 bytes
MAX_SCRIPT_SIZE	10,000 bytes
MAX_OPS_PER_SCRIPT	201
MAX_STACK_SIZE	1,000 items
MAX_SCRIPT_NUM_SIZE	4 bytes
MAX_LOCKTIME_NUM_SIZE	5 bytes
MAX_PUBKEYS_PER_MULTISIG	20
TAPSCRIPT_SIGOPS_BUDGET_BASE	50
TAPSCRIPT_SIGOPS_COST	50
TARGET_SPACING	600 seconds
TARGET_TIMESPAN	1,209,600 seconds
DIFFICULTY_ADJUSTMENT_INTERVAL	2,016 blocks
MAX_RETARGET_FACTOR	4
MAX_FUTURE_BLOCK_TIME	7,200 seconds
BIP94_TIMEWARP_GRACE	600 seconds

These constants are specified by deployed BIPs and Bitcoin Core consensus [10, 12, 8, 13, 17, 32, 6].

Appendix B. Mainnet Parameters

This appendix collects network parameters and historical consensus exceptions that are inputs to $\text{Rules}(N, C, h, t)$. Mainnet is the default network for this specification.

B.1 Mainnet parameters

Parameter	Value
Genesis hash	000000000019d6689c085ae165831e93 4ff763ae46a2a6c172b3f1b60a8ce26f
Genesis Merkle root	4a5e1e4baab89f3a32518a88c31bc87f 618f76673e2cc77ab2127b7afdeda33b
Message start	f9 be b4 d9
Subsidy halving interval	SUBSIDY_HALVING_INTERVAL
Proof-of-work target spacing	TARGET_SPACING
Proof-of-work target timespan	TARGET_TIMESPAN
Bech32 human-readable part	bc

Other networks, including testnet, testnet4, signet, and regtest, have distinct genesis blocks, proof-of-work limits, retargeting exceptions, message-start bytes, ports, address prefixes, and activation parameters. Their rules are not inferred from mainnet.

B.2 Historical exceptions

Exception	Rule
Genesis coinbase	The genesis block’s coinbase transaction is part of the block history but its output is not inserted into U by normal block connection.
BIP30 duplicates	Historical duplicate-transaction cases are handled by the BIP30 no-overwrite rule and its deployed exceptions. Outside those exceptions, creating an output whose outpoint is already unspent is invalid.
Buried activations	Mainnet rules whose activation heights are buried in contemporary software are represented by $\text{Rules}(N, C, h, t)$, not by live versionbits state.
Future-dated blocks	A block whose timestamp is more than <code>MAX_FUTURE_BLOCK_TIME</code> after the node’s current clock is temporarily ineligible for active validation, not permanently invalid.

Signet uses a block signature challenge in the witness commitment payload. Testnet4 and regtest can enforce BIP94 timewarp mitigation through network parameters; mainnet does not set that parameter in the Bitcoin Core 31.0 baseline [26, 3, 12, 1, 6].

Appendix C. Remote Procedure Calls (RPC)

TODO

References

- [1] Karl-Johan Alm and Anthony Towns. Bip 325: Signet. <https://github.com/bitcoin/bips/blob/master/bip-0325.mediawiki>.
- [2] Gavin Andresen. Bip 16: Pay to script hash. <https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki>.
- [3] Gavin Andresen. Bip 34: Block v2, height in coinbase. <https://github.com/bitcoin/bips/blob/master/bip-0034.mediawiki>.
- [4] Bitcoin contributors. Bitcoin improvement proposals. <https://github.com/bitcoin/bips>.
- [5] Bitcoin Core contributors. Bitcoin core source repository, 2009. <https://github.com/bitcoin/bitcoin>.
- [6] Bitcoin Core contributors. Bitcoin core v31.0 source tree, 2026. <https://github.com/bitcoin/bitcoin/tree/v31.0>.
- [7] Bitcoin.org contributors. Bitcoin developer reference. <https://developer.bitcoin.org/reference/>.
- [8] BtcDrak, Mark Friedenbach, and Eric Lombrozo. Bip 112: Checksequenceverify. <https://github.com/bitcoin/bips/blob/master/bip-0112.mediawiki>.
- [9] Matt Corallo. Bip 152: Compact block relay. <https://github.com/bitcoin/bips/blob/master/bip-0152.mediawiki>.
- [10] Mark Friedenbach, BtcDrak, Nicolas Dorier, and Kalle Rosenbaum. Bip 68: Relative lock-time using consensus-enforced sequence numbers. <https://github.com/bitcoin/bips/blob/master/bip-0068.mediawiki>.
- [11] Mike Hearn and Matt Corallo. Bip 37: Connection bloom filtering. <https://github.com/bitcoin/bips/blob/master/bip-0037.mediawiki>.
- [12] Fabian Jahr. Bip 94: Testnet 4. <https://github.com/bitcoin/bips/blob/master/bip-0094.mediawiki>.
- [13] Thomas Kerin and Mark Friedenbach. Bip 113: Median time-past as endpoint for lock-time calculations. <https://github.com/bitcoin/bips/blob/master/bip-0113.mediawiki>.
- [14] Johnson Lau. Bip 143: Transaction signature verification for version 0 witness program. <https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki>.
- [15] Johnson Lau. Bip 147: Dealing with dummy stack element malleability. <https://github.com/bitcoin/bips/blob/master/bip-0147.mediawiki>.
- [16] Eric Lombrozo. Bip 123: Bip classification. <https://github.com/bitcoin/bips/blob/master/bip-0123.mediawiki>.
- [17] Eric Lombrozo, Johnson Lau, and Pieter Wuille. Bip 141: Segregated witness. <https://github.com/bitcoin/bips/blob/master/bip-0141.mediawiki>.

- [18] Eric Lombrozo, Johnson Lau, and Pieter Wuille. Bip 144: Segregated witness (peer services). <https://github.com/bitcoin/bips/blob/master/bip-0144.mediawiki>.
- [19] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. <https://bitcoin.org/bitcoin.pdf>.
- [20] Olaoluwa Osuntokun and Alex Akselrod. Bip 157: Client side block filtering. <https://github.com/bitcoin/bips/blob/master/bip-0157.mediawiki>.
- [21] Olaoluwa Osuntokun and Alex Akselrod. Bip 158: Compact block filters for light clients. <https://github.com/bitcoin/bips/blob/master/bip-0158.mediawiki>.
- [22] Peter Todd. Bip 65: Op_checklocktimeverify. <https://github.com/bitcoin/bips/blob/master/bip-0065.mediawiki>.
- [23] Wladimir J. van der Laan. Bip 155: addrv2 message. <https://github.com/bitcoin/bips/blob/master/bip-0155.mediawiki>.
- [24] Gavin Wood. The jam specification (graypaper source repository), 2024. <https://github.com/gavofyork/graypaper>.
- [25] Gavin Wood and Ethereum contributors. Ethereum yellow paper. <https://github.com/ethereum/yellowpaper>.
- [26] Pieter Wuille. Bip 30: Duplicate transactions. <https://github.com/bitcoin/bips/blob/master/bip-0030.mediawiki>.
- [27] Pieter Wuille. Bip 42: A finite monetary supply for bitcoin. <https://github.com/bitcoin/bips/blob/master/bip-0042.mediawiki>.
- [28] Pieter Wuille. Bip 66: Strict der signatures. <https://github.com/bitcoin/bips/blob/master/bip-0066.mediawiki>.
- [29] Pieter Wuille, Dhruv Mehta, Tim Ruffing, and Jonas Nick. Bip 324: Version 2 p2p encrypted transport protocol. <https://github.com/bitcoin/bips/blob/master/bip-0324.mediawiki>.
- [30] Pieter Wuille, Jonas Nick, and Tim Ruffing. Bip 340: Schnorr signatures for secp256k1. <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>.
- [31] Pieter Wuille, Jonas Nick, and Anthony Towns. Bip 341: Taproot: Segwit version 1 spending rules. <https://github.com/bitcoin/bips/blob/master/bip-0341.mediawiki>.
- [32] Pieter Wuille, Jonas Nick, and Anthony Towns. Bip 342: Validation of taproot scripts. <https://github.com/bitcoin/bips/blob/master/bip-0342.mediawiki>.
- [33] Zcash contributors. Zcash protocol specification. <https://zips.z.cash/protocol/protocol.pdf>.